

Reinforcement Learning for Knowledge Graph Co-optimization: A Dual-Strategy Rule Generation Framework

Anonymous Authors

Abstract—Knowledge-graph quality control couples two decisions: which constraints to acquire and which graph defects to repair under a limited budget. We formulate this interaction as a finite-horizon control problem and train a Double-DQN policy over four graph operations and four rule-management operations. A dual-strategy module proposes text-derived rule candidates through deletion completion and augmentation expansion. The reward measures introduced violations by identity after each edit. On a controlled-corruption benchmark, ten retrained Double-DQN policies reach mean normalized final quality 0.96855 and introduce no new violations. An acquire-then-deficit heuristic reaches 0.98248 but introduces 4.8 new violations per run; mean discounted returns are 0.3660 and 0.3508, respectively. Forty retrained ablation models support the action mask’s contribution to final quality, while other component gains do not pass multiplicity correction. Replaying 9,456 archived responses shows complementary candidates on the same documents, but augmentation alone yields more constraint candidates at equal call budgets. An execution audit compiles 18,143 exact typed patterns; their union detects 10 of 64 defects in the designed RuleTest-94 suite. A separate offline bridge activates these rules and removes ten prohibited records while retaining all thirty clean records. The results characterize the tradeoff between structural improvement and new violations, and distinguish candidate diversity, executable coverage and factual restoration.

I. Introduction

A. Research Background and Motivation

Knowledge graph (KG) quality control involves two linked decisions: which defects to repair and which rules to acquire. A repair may require a validator that is not yet available. Acquiring more rules consumes resources, and a rule that covers few remaining defects may have little immediate value. A fixed sequence of rule generation and graph correction cannot use this changing balance.

Rules also require a source of new candidates. Expert constraints capture known requirements, while text can suggest additional type, hierarchy, and procedural constraints. Large language models (LLMs) offer a way to propose such candidates. Turning a proposal into a useful validator requires executable semantics, a compatible vocabulary, and evidence of detection quality.

We study reinforcement learning (RL) for allocating a finite budget between graph and rule operations, together with two complementary ways to elicit rule candidates from text. The scheduling experiment uses a controlled environment with a fixed validator registry. The generation experiment uses

archived LLM responses. This separation lets us inspect policy behavior and rule content with reproducible inputs.

B. Contributions of This Paper

We make three contributions:

- 1) **An executable graph–rule co-optimization environment.** We define fourteen observed features, eight graph and rule actions, feasibility masks, and a cost-aware reward. Graph actions change stored records, and rule actions change the validators available for later repairs.
- 2) **A trained Double-DQN policy and matched control study.** We archive independently trained policies, replay histories, and paired evaluation transitions. Feasible-action baselines and four retrained ablations test the roles of observed graph features, observed rule features, action masking, and acquisition cost. The corrected reward exposes a tradeoff: Double DQN introduces fewer new violations, while a simple acquire-then-deficit heuristic reaches higher structural quality.
- 3) **Dual-strategy rule generation with traceable evaluation.** Deletion completion and augmentation expansion propose candidates in a shared schema. We compare candidate diversity at equal call budgets, distinguish declarations from constraints, and trace exact typed-rule execution back to the archived response. This identifies both complementary proposals and the vocabulary gap between generated patterns and the test suite.

Figure 1 summarizes the framework. The policy receives fourteen features describing graph quality, rule quality, remaining defects, remaining budget, and active rule fractions. A feasibility mask controls action selection. The selected operation changes the environment, which recomputes quality and reward. A separate model-informed reference probes possible one-step transitions; it is a diagnostic comparator, not a proven upper bound.

C. Paper Organization

Section II reviews related work. Section III presents the scheduling model and rule-generation procedures. Section IV reports the experiments, followed by conclusions and reproducibility details.

Budgeted graph–rule co-optimization

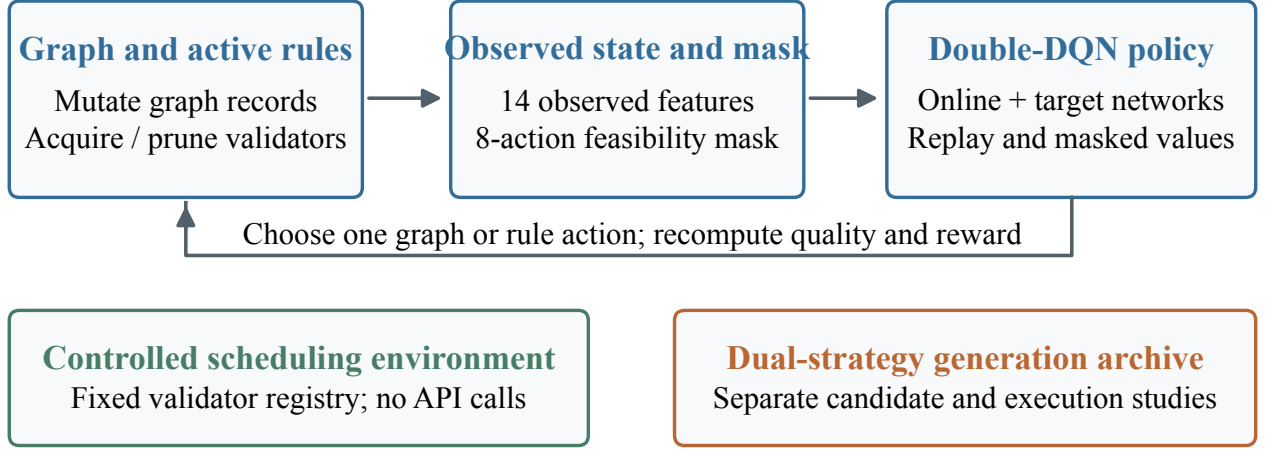


Fig. 1. RL graph–rule co-optimization. The controlled scheduling loop uses a fixed validator registry. The text-derived candidate archive is evaluated through the dual-strategy analysis and direct execution audit.

II. Related Work

A. Reinforcement Learning for Knowledge Graphs

MINERVA by Das et al. [1] learns policies for multi-hop query answering. DeepPath by Xiong et al. [2] uses reinforcement learning to discover reasoning paths. Lin et al. [3] study reward shaping for multi-hop reasoning. These methods select graph traversal actions to answer queries. Our action space instead changes graph records and acquires validators that enable later repairs.

The scheduling question is whether a learned value function can allocate a limited budget better than a fixed schedule or an observation-based heuristic. We compare these alternatives on the same executable transitions and use retrained ablations to test the roles of observed features and action masking. The controlled registry lets us reproduce each transition, while the text-derived rule study examines the candidate source separately.

B. Language Models for Knowledge Engineering

KG-BERT [4] uses a language model for triple classification, and KEPLER [5] combines language modeling with knowledge embeddings. Pan et al. [6] review ways to connect language models and knowledge graphs. These studies motivate using text as a source of information for graph construction and completion.

Chain-of-thought prompting [7] and ReAct [8] explore structured reasoning and tool use. Our generation module uses two single-call elicitation tasks with a common output schema: deletion asks which constraints are exposed by missing spans, while augmentation asks which constraints are suggested by supplementary clauses. We evaluate the resulting candidates through archived provenance and executable patterns, rather than treating a reasoning response as a validated rule.

C. Constraint Languages, Rule Mining, and Repair

SHACL [9] and ShEx [10] express graph constraints. Their detection coverage depends on the shapes provided. A limited set of schema checks should therefore be distinguished from the expressiveness of the constraint language itself.

AMIE [11] mines Horn rules from graph facts, RuDiK [12] studies rule discovery in knowledge bases, and Neural LP [13] learns logical rules through differentiable operations. Our candidates originate from source documents and include type patterns and procedural statements. Their different representation requires a validation and compilation step before they can act as graph validators.

Lin et al. [14] systematically evaluate LLM-based graph repair. This makes controlled constraint and repair comparisons directly relevant to our study. We focus on the order of validator acquisition and graph edits, with an additional provenance-based audit of generated candidates. An end-to-end comparison with external repair and rule-mining methods requires aligning executable constraints, input evidence, and evaluation budgets.

III. Methodology

A. Framework Overview

The framework maintains a graph G_t and a rule set R_t . At each step, a policy chooses whether to acquire or prune rules, or to repair the graph using an active validator. Rule acquisition can enable later repairs, while the remaining graph defects determine the value of further acquisition. Our research question concerns this allocation of a finite decision budget.

We implement the scheduling benchmark with graph mutations and a fixed registry of executable validators. The dual-strategy module supplies a separate archive of text-derived

candidates. We evaluate candidate diversity, a directly executable subset, and scheduling separately. The controlled policy environment uses its registered validators; a separate offline bridge activates exact archived rules on a vocabulary-matched test suite. Figure 1 distinguishes these execution paths.

B. Quality Assessment Model

1) *Graph Quality*: After every graph operation, deterministic checks compute isolation, exact-duplicate relation, schema-invalid relation, and dangling-edge rates, denoted I_r, D_r, V_r, L_r . Their quality components are

$$\begin{aligned} S(C_1) &= 1 - I_r, & S(C_2) &= 1 - D_r, \\ S(C_3) &= 1 - V_r, & S(C_4) &= 1 - L_r. \end{aligned} \quad (1)$$

The controlled environment uses

$$Q(G) = 0.20S(C_1) + 0.20S(C_2) + 0.30S(C_3) + 0.30S(C_4). \quad (2)$$

These fixed weights define the benchmark objective. They do not estimate the factual correctness of arbitrary triples.

2) *Rule Quality*: Active validators are scored on a fixed 180-case calibration inventory, separate from the graph corruption manifest. Their union prediction determines

$$\text{Precision}(R) = \frac{TP}{TP + FP}, \quad \text{Recall}(R) = \frac{TP}{TP + FN}. \quad (3)$$

Both are zero for an empty active set in the environment. Coverage is the fraction of four registered defect families with an active validator. The online score is

$$\begin{aligned} Q_{\text{online}}(R) &= 0.35 \text{Precision}(R) + 0.35 \text{Recall}(R) \\ &\quad + 0.30 \text{Coverage}(R). \end{aligned} \quad (4)$$

The calibration predictions are prescribed by the controlled benchmark: each high-precision module detects 27 of 30 family positives and two clean cases. Three additional modules provide lower-precision alternatives. These scores describe that inventory, rather than estimates from a live production stream. Candidate counts from the generation archive are measured separately from rule precision and recall.

C. RL-Based Graph–Rule Co-Optimization

1) *Environment and Policy Observation*: The underlying finite-horizon state contains the graph, active rules, remaining budget, initial defect counts, and the environment’s trusted relation-restoration records. These records are private to the environment. The policy receives a 14-dimensional observation:

$$\begin{aligned} o_t &= [S(C_1), S(C_2), S(C_3), S(C_4), \\ &\quad \text{Precision}, \text{Recall}, \text{Coverage}, \tilde{n}_{\text{iso}}, \tilde{n}_{\text{dup}}, \\ &\quad \tilde{n}_{\text{invalid}}, \tilde{n}_{\text{dang}}, \tilde{b}, \rho_{\text{valid}}, \rho_{\text{noisy}}]_t. \end{aligned} \quad (5)$$

Violation counts are divided by their episode-initial counts, with denominator at least one. The final terms give the remaining budget fraction and active fractions of the predefined high- and low-precision modules. The observation is clipped to $[0, 1.5]$ componentwise. Coverage and ρ_{valid} coincide in this

registry; both entries are retained for matched architectural comparisons. This compressed observation need not be a sufficient Markov state. The network uses it as a value-function input, together with an externally computed feasible-action mask.

The eight actions are

$$\mathcal{A} = \{a_{\text{iso}}, a_{\text{dup}}, a_{\text{rel}}, a_{\text{dang}}, a_{\text{del}}, a_{\text{aug}}, a_{\text{mix}}, a_{\text{prune}}\}. \quad (6)$$

The first four remove isolated noise nodes, deduplicate relations, restore invalid relation names, and remove dangling edges. Each processes up to a batch determined by 60% of the remaining matching defects. Relation restoration uses the private corruption record after detection. The other actions activate a deletion-family module, activate an augmentation-family module, activate one available module from each family, or prune modules with calibration precision below 0.5. Module acquisition follows the fixed registry order. A repair is feasible only if its validator is active and matching defects remain.

The mask is computed from the current graph and active rule identities. It is not necessarily recoverable from the fourteen observation values alone. All matched policy baselines receive the same mask, except the explicit no-mask ablation. Policies cannot access the private corruption record or query future transitions; the separate model-informed reference can clone and probe the environment.

2) *Reward and Double DQN*: The reward uses $\Delta Q(G_t) = Q(G_{t+1}) - Q(G_t)$ and the analogous rule-score change, with costs for acquisition, edits, and new violations:

$$\begin{aligned} r_t &= \Delta Q(G_t) + 0.4 \Delta Q_{\text{online}}(R_t) \\ &\quad - 0.004 C_{\text{call}} - 2 \times 10^{-5} C_{\text{edit}} - 0.01 N_{\text{new}}. \end{aligned} \quad (7)$$

C_{call} counts one unit for either single-family acquisition and two for mixed acquisition. It is an accounted model-call cost, not an actual API request. Pruning has no call cost. The revised environment computes N_{new} from violation identities before and after the action, rather than returning a fixed zero. Nodes have persistent identifiers; each edge occurrence has its own identifier, including duplicate occurrences. The edit counter includes graph mutations and changes in active rules; we also report these two components separately. An episode ends when $Q(G) \geq 0.999$ and $Q_{\text{online}}(R) \geq 0.90$, after 18 decisions, or when no feasible action remains. A destructive proposal that would clear a nonempty node or edge set is rolled back and ends the episode.

Let $\mathcal{V}(G)$ be the multiset of detected violation identities. Then

$$N_{\text{new}} = |\mathcal{V}(G_{t+1}) \setminus_m \mathcal{V}(G_t)|, \quad (8)$$

where $\setminus_m$ is multiset difference. Removing a dangling edge can isolate its remaining endpoint: one old violation disappears and a different one appears, although the total count stays unchanged. The new term charges for that introduced violation. Each transition stores the introduced identities and all reward

components. For $J_t = Q(G_t) + 0.4Q_{\text{online}}(R_t)$, discounted quality gains satisfy

$$\sum_{t=0}^{T-1} \gamma^t (J_{t+1} - J_t) = -J_0 + \gamma^{T-1} J_T + (1 - \gamma) \sum_{t=1}^{T-1} \gamma^{t-1} J_t. \quad (9)$$

Thus the policy values earlier improvements as well as terminal quality.

We use DQN [15] with the Double-DQN target [16]. The online Q-network has architecture $14 \rightarrow 32 \rightarrow 16 \rightarrow 8$ with ReLU activations. A replay buffer stores $(o_t, a_t, r_t, o_{t+1}, d_t)$ and the next-action mask. For a nonempty next-action set, a^* selects the online-network maximizer. The target is

$$a^* = \arg \max_{a' \in \mathcal{A}_{t+1}} Q(o_{t+1}, a'; \theta),$$

$$y_t = \begin{cases} r_t, & d_t = 1 \text{ or } \mathcal{A}_{t+1} = \emptyset, \\ r_t + \gamma Q(o_{t+1}, a^*; \theta^-), & \text{otherwise,} \end{cases}$$

$$\mathcal{L}(\theta) = \mathbb{E}[\text{Huber}(Q(o_t, a_t; \theta) - y_t)]. \quad (10)$$

The bootstrap term is zero for terminal states or an empty feasible set. Target parameters are copied every 100 environment steps. Standard DQN changes only the next-state target to $\max_{a'} Q(o_{t+1}, a'; \theta^-)$.

We use Adam with learning rate 0.002, discount 0.95, replay capacity 20,000, batch size 64, and 128 transitions of warm-up. Each model trains for 250 episodes. The archived configuration decays exploration linearly over 260 episodes, so the last training episode uses $\epsilon \approx 0.0902$, rather than reaching the lower limit of 0.05. Ten independently trained checkpoints are retained for each of the six learned settings in the corrected study. Training uses the local environment and makes no API calls.

D. LLM-Based Dual-Strategy Rule Generation

The two strategies propose source-based candidates that can extend a rule inventory. The stored implementation uses one generation call per document and strategy. Its JSON response contains entity-type declarations, relation-type declarations, allowed and forbidden type-relation patterns, hierarchy statements, geographical hierarchy patterns, and procedural statements. Declarations and constraint candidates are counted separately in the matched-budget analysis.

1) *Deletion Completion*: The implementation removes short character spans, rather than entity-level masks. It selects up to five start points, with each span at most six characters under the script defaults, and replaces them with a missing-span marker. The prompt supplies the original text, modified text, removed fragments, and a shared JSON schema. It asks the model to describe missing rule information and propose reusable constraints. Providing the original text makes this an explicit contrastive elicitation task; it is not a blind reconstruction test.

Algorithm 1 Archived deletion-completion procedure

Require: Document identifier d , source text T

- 1: $(T^-, F) \leftarrow \text{DeleteShortSpans}(T)$
 - 2: $P \leftarrow \text{DeletionPrompt}(T, T^-, F)$
 - 3: $Y \leftarrow \text{LLM}(P)$ ▷ One call
 - 4: Parse the JSON candidate categories in Y
 - 5: Log $(d, \text{deletion}, F, Y)$
 - 6: **return** categorized candidates, including parse status
-

Algorithm 2 Archived augmentation-expansion procedure

Require: Document identifier d , source text T

- 1: $P \leftarrow \text{AugmentationPrompt}(T)$
 - 2: $Y \leftarrow \text{LLM}(P)$ ▷ Clauses and candidates in one call
 - 3: Parse the JSON candidate categories in Y
 - 4: Log $(d, \text{augmentation}, Y)$
 - 5: **return** categorized candidates, including parse status
-

2) *Augmentation Expansion*: The augmentation prompt supplies the source and asks for up to three plausible supplementary clauses under the script default. The same response proposes constraints suggested by those clauses. It does not run a separate graph extractor on each variation. The generated clauses provide exploration cues; they are not assumed to be independent samples from the real document distribution.

3) *Rule Materialization and Validation*: A candidate is not automatically an executable or correct rule. We preserve the source document, strategy, response line, candidate category, and ordinal position. The direct execution audit supports explicit triples of subject type, relation, and object type. A forbidden pattern $r = (\tau_s, p, \tau_o)$ flags a record x when

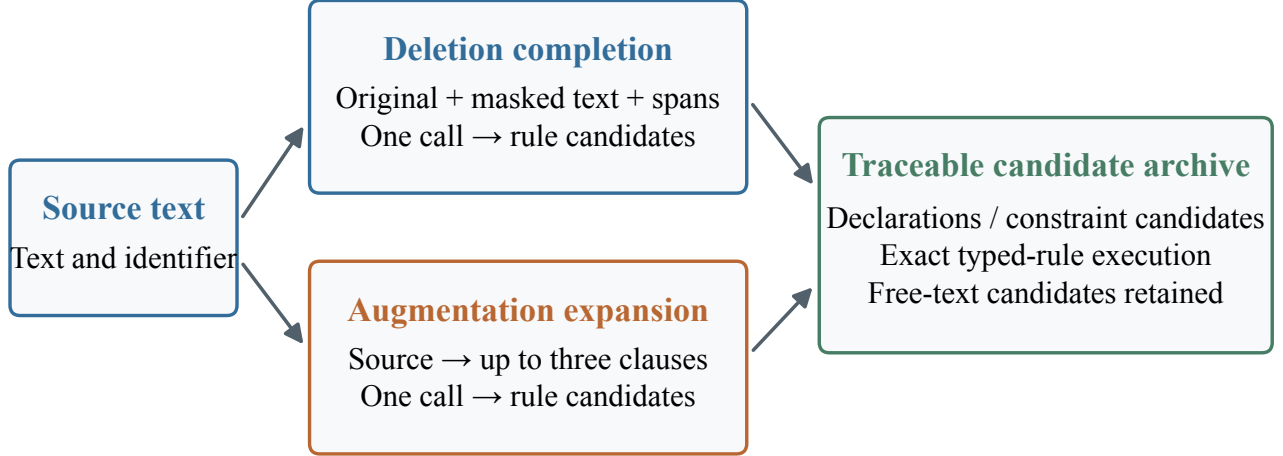
$$V_r(x) = \mathbf{1}[(\text{type}(s_x), p_x, \text{type}(o_x)) = r]. \quad (11)$$

An allowed pattern records permission for that exact combination. Absence from the allowed set does not imply a violation. When both allowed and forbidden patterns match, the audit records a conflict and abstains. Only outer whitespace is trimmed; no aliases or semantic rewrites are inferred.

Each malformed candidate, unsupported family, unmapped vocabulary item, and unmatched executable pattern remains in the audit. The executable subset is checked on RuleTest-94 using its stored expected outcomes. These labels describe the designed suite and provide an execution check, not independent adjudication of generated rule meaning. The procedural and structural family detectors are a separate hand-implemented comparison and are not attributed to individual generated candidates.

4) *Aggregation and Provenance*: Candidates are deduplicated within category by the stored aggregation code and canonical serialization. Entity and relation declarations are distinguished from constraints. The replay uses category-level canonical deduplication; it makes no additional semantic merges.

The generation script defaults to Qwen3-32B and temperature 0.2. Per-call records do not preserve a complete run configuration, exact rendered prompt, or decoding metadata. We therefore report those values as script defaults, not verified



Equal-call comparison: B documents for one strategy; B/2 documents for both

Fig. 2. Dual-strategy generation and the offline execution audit. Both strategies return the same candidate categories. Exact typed constraints can be executed directly; free-text candidates remain in the archive for further validation.

settings for every historical call. The prompt builders and translated templates are documented in Appendix A; literal templates are included in the supplementary artifact. The independent extraction benchmark uses Qwen3.8-27B-no-thinking and is separate from rule generation.

E. Offline Generated-Rule Bridge

The bridge compiles archived allowed and forbidden typed patterns using the same exact-match semantics as the execution audit. Activation changes the current violation report and enables a removal action when an unopposed forbidden pattern matches a record. After removal, the bridge recomputes matches and action availability. Every match retains its rule identifier and source lineage. Unsupported candidates are logged; conflicting permissions and prohibitions cause abstention.

This prototype uses government-domain typed records rather than mapping the archive’s vocabulary to TNEWS. Its fixed acquisition schedule exercises the interfaces without training another policy. It removes prohibited records but does not infer replacement facts; reference labels are used only to count defective and clean removals after execution.

IV. Experiments

A. Evaluation Design

1) *Research Questions*: We ask three main questions. **RQ1**: How does a learned co-optimization policy compare with feasible-action schedules and a simple adaptive heuristic? **RQ2**: Which observation features and training components affect its decisions? **RQ3**: What do the two generation strategies contribute at equal call budgets, and which archived candidates can execute on a labeled suite? Additional structural and extraction tests describe the behavior of the repair layer.

2) *Data and Protocol*: The policy environment uses 450 balanced TNEWS/CLUE documents, 30 per category, to build a graph with 873 nodes and 1,083 relations. Ten scenario seeds control isolation, duplication, invalid-relation, and dangling-edge corruption. The separate 180-case calibration inventory supplies online validator precision and recall. Rule acquisition activates registered modules; relation restoration uses the environment’s private source record after a validator detects an invalid relation.

The rule study replays 9,456 successful archived responses, with deletion and augmentation responses for each of 4,728 government-service documents. RuleTest-94 supplies 30 clean and 64 defective records with stored expected outcomes. It is a designed test suite. A deterministic repair test uses 3,000 TNEWS titles, and an earlier API extraction test uses another recorded 45-title sample. These tests measure different parts of the implementation.

3) *Reproducibility*: The mathematical revision preserves the original environment and all earlier checkpoints. It freezes a corrected environment with violation-identity accounting, retrains DQN, Double DQN, and four ablations (sixty models), and archives all 110 evaluation trajectories. Training uses ten seeds and 250 episodes per model. Evaluation reuses the ten original scenarios, allowing paired follow-up comparisons. All new runs make zero API requests. The earlier extraction test used Qwen3.8-27B-no-thinking; the rule archive’s historical decoding metadata is described in Appendix A.

B. RL Co-optimization on Executable Graph Transitions

1) *Matched Policies and Measures*: Random, alternating, and rule-first schedules now select only feasible actions. They receive the same observation and mask as the learned methods. The stronger *acquire-then-deficit* heuristic acquires missing high-precision modules in mixed, deletion, then augmentation

priority. Once all four are active, it selects the feasible graph action with the largest weighted current quality deficit, using the weights in Equation 2. It reads no private corruption record and probes no transitions. This is a fixed observation-based heuristic, with no fitted thresholds.

A separate model-informed lookahead policy clones the environment and evaluates feasible one-step transitions before acting. We report its probe count to distinguish this information advantage. A one-step choice is not an upper bound on finite-horizon performance.

Final joint quality and common-budget trajectory quality are

$$\begin{aligned}\bar{Q}_t &= \frac{Q(G_t) + 0.4Q_{\text{online}}(R_t)}{1.4}, \\ \text{AUC}_{18} &= \frac{1}{18} \sum_{t=0}^{17} \frac{\bar{Q}_t + \bar{Q}_{t+1}}{2}.\end{aligned}\quad (12)$$

An early terminal score is carried forward through step 18. All policies share this horizon and objective. Calls denote acquisition cost units in the environment; actual API requests are zero. Invalid actions, graph edits, rule edits, environment probes, and local elapsed time are recorded separately. The new AUC definition replaces the earlier archive’s duration-normalized trajectory average for the matched comparison.

2) *Policy Results*: With the corrected reward, Double DQN reaches mean final quality 0.96855 and AUC 0.94514, using 6.2 accounted calls. The acquire-then-deficit heuristic reaches 0.98248 and 0.95797 with four calls. Full Double DQN minus the heuristic gives differences of -0.01392 and -0.01283 ; both Holm-adjusted p values are 0.0195 (Table III). Standard DQN reaches final quality 0.96920.

The reward now charges for introduced violations. Double DQN introduces none on these ten evaluation runs, while the heuristic introduces 4.8 per run on average. For $\mathcal{R} = \sum_{t=0}^{T-1} 0.95^t r_t$, the mean discounted returns are 0.3660 and 0.3508, respectively. These descriptive measures show why final structural quality and the training objective give different rankings: avoiding newly introduced violations can leave some existing defects unresolved. We report both, rather than replacing the quality comparison with return. All reward components and introduced identities are retained for each transition.

3) *Retrained Ablations*: We train ten models for each of four variants, using the original 250 episodes per seed. The variants remove explicit graph features, explicit rule features, the action mask, or the acquisition-call penalty. Feature removal applies during both training and evaluation. The mask remains in the feature ablations and can convey rule availability. The no-mask variant removes masking from action selection and Bellman targets. The no-call-penalty variant changes only the training reward; evaluation uses the same corrected reward and cost coefficients for all settings.

Removing the mask lowers final quality to 0.94800 and produces 10.7 invalid actions per run. Its final-quality contrast passes Holm correction ($p = 0.0195$); its AUC contrast does not ($p = 0.8340$). The graph-feature, rule-feature, and call-penalty contrasts do not pass the ten-test correction. Thus the revised results support the action mask’s contribution to final quality, while the other independent gains remain unestablished in this environment.

The analysis enumerates all 2^{10} sign assignments for exact two-sided paired randomization tests and applies Holm correction across ten contrasts: full Double DQN against the heuristic and four ablations, for both metrics. Pointwise intervals use 10,000 paired bootstrap draws. The sixty new models contribute 15,000 completed training episodes. Earlier checkpoints and results remain archived under their original environment version.

C. Dual-Strategy Candidate Generation

1) *Same-Document and Equal-Call Comparisons*: On the full paired archive, deletion yields 32,955 distinct categorized candidates, augmentation yields 41,683, and their union contains 71,656. These totals include entity and relation declarations. The union is 71.9% larger than augmentation alone but uses twice as many calls: 9,456 versus 4,728. It measures additional candidate coverage on the same documents.

To compare candidate diversity at equal cost, we fix call budgets of 100, 500, 1,000, 2,000, and 4,000. For each of thirty seeded permutations of the paired document inventory, a single strategy processes B documents once each; the dual strategy processes the first $B/2$ documents with both strategies. Thus call count is matched while document coverage differs. Canonical aggregation follows the archived code, with declarations separated from constraint candidates. Table IV gives representative budgets; all budgets and samples are released.

Augmentation produces more distinct constraint candidates than the dual strategy at every matched budget. At 1,000 calls, the means are 7,787 versus 7,126. Dual generation provides complementary proposals for a given document set, but it does not show a candidate-count advantage per call in this archive. Candidate correctness is evaluated separately from diversity.

2) *Provenance-Based Execution Audit*: The archive contains 181,851 candidate occurrences. Exact allowed/forbidden type patterns compile to 18,143 distinct rules, retaining the document, strategy, response line, category, and ordinal of each occurrence. Only two distinct compiled patterns match any RuleTest-94 record. Of 32,360 compilable occurrences, 32,348 contain vocabulary absent from the relevant suite positions, nine use represented vocabulary but match no record, and three match a record. This exposes a schema-alignment gap between the source archive and the test suite.

Direct execution of the union detects 10 of 64 defects and flags none of the 30 clean cases, giving recall 0.156 and F1 0.270. Augmentation alone makes no positive prediction, so its precision is undefined. Allowed patterns act as permissions rather than a closed-world whitelist; conflicting permissions and prohibitions cause abstention. No conflicts occur on this suite. Unsupported and malformed candidates remain in the audit counts and provenance files. Each strategy is scored on all 94 test cases.

3) *Generated Rules in an Offline Repair Loop*: The bridge in Section III-E activates the exact archived typed patterns on RuleTest-94, recomputes detections and the removal-action mask, executes allowed removals, and records the next state. Deletion activates 5,794 patterns, augmentation 12,617, and

TABLE I
Corrected-environment policy comparison over ten paired scenarios (mean $\pm$ sample standard deviation). Calls are accounted acquisition units.

Policy	Final quality	AUC ₁₈	Calls	Invalid	New viol.	Return
Random valid	0.9642 $\pm$ 0.0065	0.9254 $\pm$ 0.0152	10.2	0.0	2.9	0.3163
Alternating valid	0.9677 $\pm$ 0.0007	0.9292 $\pm$ 0.0020	10.0	0.0	4.0	0.3102
Rule-first valid	0.9747 $\pm$ 0.0004	0.9365 $\pm$ 0.0024	8.0	0.0	4.6	0.3255
Acquire + deficit	0.9825 $\pm$ 0.0001	0.9580 $\pm$ 0.0014	4.0	0.0	4.8	0.3508
DQN	0.9692 $\pm$ 0.0037	0.9454 $\pm$ 0.0079	5.2	0.0	0.0	0.3688
Double DQN	0.9686 $\pm$ 0.0034	0.9451 $\pm$ 0.0096	6.2	0.0	0.0	0.3660
Model-informed lookahead	0.9711 $\pm$ 0.0048	0.9516 $\pm$ 0.0027	5.1	0.0	0.2	0.3750

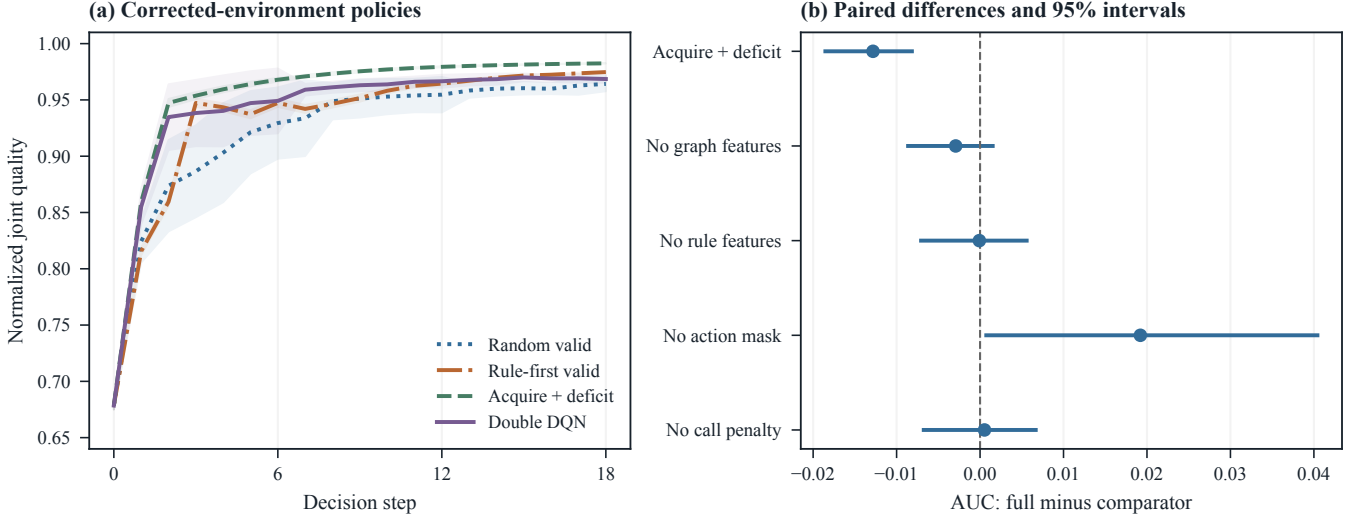


Fig. 3. Matched policy study. (a) Mean evaluation quality with one-standard-deviation bands over ten scenarios. Terminal scores are carried forward to step 18. (b) Full Double DQN minus each comparator in AUC, with pointwise 95% paired-bootstrap intervals. Multiplicity-adjusted tests appear in Table III.

TABLE II
Corrected-environment ablations, ten retrained models per setting.

Setting	Final quality	AUC ₁₈	Calls	Invalid	New viol.	Return
Double DQN	0.9686 $\pm$ 0.0034	0.9451 $\pm$ 0.0096	6.2	0.0	0.0	0.3660
No graph features	0.9713 $\pm$ 0.0041	0.9481 $\pm$ 0.0028	5.1	0.0	0.9	0.3693
No rule features	0.9692 $\pm$ 0.0062	0.9452 $\pm$ 0.0034	7.6	0.0	0.5	0.3594
No action mask	0.9480 $\pm$ 0.0215	0.9259 $\pm$ 0.0278	4.9	10.7	0.0	0.3430
No call penalty	0.9659 $\pm$ 0.0051	0.9446 $\pm$ 0.0044	8.1	0.0	0.0	0.3596

their union 18,143. Deletion and the union each remove ten defective records and no clean record; augmentation changes no record. All thirty clean records remain. The deleted records have no inferred replacements, so this is constraint removal rather than factual reconstruction.

These runs use the previously inspected designed suite and a fixed acquisition schedule. They demonstrate a working archive-to-action interface, separate from the TNEWS RL comparison. Synthetic interface tests additionally check conflict abstention, malformed candidates and unsupported families without adding those probes to the empirical sample count.

4) *Separate Rule-Family Check*: The hand-implemented deletion-family detector covers 34 procedural, missing-field, and specialist defects. The augmentation-family detector covers the other 30 structural defects. Their union detects all 64 defects with no false positives when scored against the stored

expected labels. These functions encode the designed defect families directly. Their coverage establishes a regression check for those families; the direct execution results in Table V measure the archived generated patterns.

D. Comparison with Constraint-Checking Baseline

We compare with a deterministic constraint-checking baseline implemented with SHACL-style RDF validation. This baseline receives deduplicated RDF triples and checks schema-conformance constraints, but it does not access the source text, LLM prompts, or generated procedural/value rules. This comparison measures the coverage of the selected shapes on these inputs.

The inputs conform to the selected shapes after RDF deduplication, so the baseline removes no additional triples. Its average quality score is 71.85 under the separate quality metric. These shapes do not implement all source-dependent and

TABLE III

Full Double DQN minus each comparator in the corrected environment. Intervals are pointwise paired-bootstrap 95% intervals; Holm adjusts ten tests.

Comparator	Metric	Difference	95% interval	P_{Holm}
Acquire + deficit	Final	-0.01392	[-0.01604, -0.01205]	0.0195
Acquire + deficit	AUC	-0.01283	[-0.01853, -0.00817]	0.0195
No graph features	Final	-0.00273	[-0.00601, -0.00007]	0.8340
No graph features	AUC	-0.00292	[-0.00862, +0.00151]	1.0000
No rule features	Final	-0.00068	[-0.00406, +0.00268]	1.0000
No rule features	AUC	-0.00008	[-0.00706, +0.00558]	1.0000
No action mask	Final	+0.02055	[+0.00897, +0.03502]	0.0195
No action mask	AUC	+0.01921	[+0.00075, +0.04042]	0.8340
No call penalty	Final	+0.00269	[-0.00031, +0.00572]	0.8340
No call penalty	AUC	+0.00053	[-0.00674, +0.00667]	1.0000

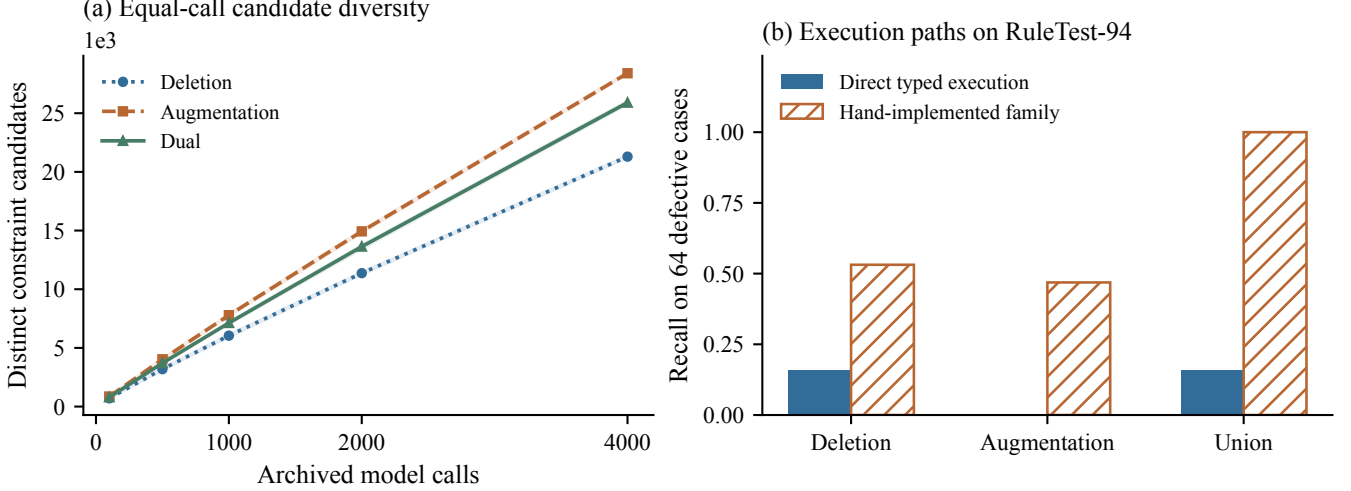


Fig. 4. Rule-generation analysis. (a) Distinct constraint candidates at equal call budgets; shading gives the empirical 2.5–97.5 percentiles across thirty document permutations, not an independent-sample confidence interval. (b) Recall from direct candidate execution and from the separate hand-implemented family detectors.

TABLE IV

Equal-call candidate comparison (means over 30 document permutations). Declarations are entity and relation types; constraints are candidate patterns or statements.

Calls	Strategy	Docs	Decl.	Constraints
100	Deletion	100	468	702
100	Augmentation	100	501	857
100	Dual	50	512	795
1000	Deletion	1000	2634	6044
1000	Augmentation	1000	2887	7787
1000	Dual	500	2970	7126
4000	Deletion	4000	7249	21299
4000	Augmentation	4000	7627	28407
4000	Dual	2000	8081	25901

procedural checks used by that metric. This is a difference in implemented constraint coverage; SHACL can express richer value and logical constraints than this baseline uses.

E. Deterministic Structural Repair

A separate deterministic benchmark uses TNEWS/CLUE-style Chinese news titles. A deterministic extractor constructs a document–category–entity KG from 3,000 balanced news titles across 15 categories; we then inject duplicate triples,

TABLE V

Direct execution of archived typed candidates on RuleTest-94. There are 64 defective and 30 clean cases.

Strategy	Patterns	TP	FP	Recall	F1
Deletion	5794	10	0	0.156	0.270
Augmentation	12617	0	0	0.000	0.000
Dual	18143	10	0	0.156	0.270

invalid relations, and isolated nodes, and run the local repair pipeline. This benchmark is fully local and makes no LLM/API calls, so it evaluates KG quality control and rule execution rather than LLM extraction quality.

The repair pipeline removes injected invalid relations, duplicate triples, and isolated noise nodes, improving structural Q from 94.93 to 100.00. The repaired graph has fewer nodes and relations than the clean extraction, so this score does not mean that every original fact was recovered. The full report is generated by `exps/external_benchmark_runner.py`.

F. API-Backed LLM Extraction Benchmark

The local TNEWS/CLUE benchmark above isolates the quality-control layer under deterministic extraction. To additionally test the full extraction path, the archived API

TABLE VI

Offline generated-rule bridge on the previously inspected RuleTest-94 suite. Removal resolves a typed prohibition; it does not reconstruct a reference fact.

Packet	Rules	Defect removals	Clean removals	Retained
Deletion	5794	10	0	84
Augmentation	12617	0	0	94
Union	18143	10	0	84

TABLE VII

Hand-implemented family detectors, rescored against stored suite labels. These detectors are separate from the compiled candidate patterns.

Detector	TP	FP	FN	Recall	F1
Deletion-family	34	0	30	0.531	0.694
Augmentation-family	30	0	34	0.469	0.638
Family union	64	0	0	1.000	1.000

TABLE VIII

SHACL-style structural validation baseline on three domains.

Domain	Raw Triples	RDF Dedup.	Removed	Q
Gov. Affairs	13,892	10,771	0	79.87
Finance	2,264	539	0	70.42
Env.	2,345	378	0	65.25
Average	6,167	3,896	0	71.85

benchmark evaluated on 45 TNEWS/CLUE titles covering 15 categories. The model is Qwen3.8-27B-no-thinking. TNEWS category labels are used as silver labels for category inference, and provided title keywords are used as weak silver mentions for entity coverage. The test measures parsing, weak extraction indicators, and structural filtering of model outputs.

The 45 calls completed in 1,190.90 seconds and all responses parsed successfully. Qwen3.8-27B-no-thinking followed the allowed schema on every sampled title, so the conservative repair layer made no change: raw and repaired structural quality are both 100.00. No structural violation was detected on this sample. Category accuracy is 0.556 and weak keyword recall is 0.178, indicating that semantic extraction remains the larger bottleneck; deterministic structural checks cannot repair omitted or misclassified content.

G. Runtime and Cost Accounting

The original twenty-model RL run recorded 1,278.99 seconds of CPU time with zero API calls. The sixty corrected-environment runs also make zero requests; their per-run durations and all evaluation timings are archived. Evaluation timing covers policy inference and environment transitions, excluding environment construction. Model-informed lookahead additionally includes cloned transition probes. These local timings do not estimate LLM latency.

The earlier 45-document extraction test made 45 actual calls and recorded 1,190.90 seconds. The rule archive records 9,456 successful responses; historical latency, tokens, retries, and failed requests are not fully recorded. Equal-call candidate comparisons therefore measure diversity per archived successful call, rather than total billed cost. The environment’s acquisition units model that cost during scheduling.

TABLE IX

TNEWS/CLUE local end-to-end KG quality benchmark.

Stage	Nodes	Relations	Invalid	Q
Clean extraction	10,871	20,827	0.000	99.89
Corrupted KG	11,414	22,909	0.050	94.93
Repaired KG	10,822	19,799	0.000	100.00

TABLE X

API-backed LLM extraction benchmark on TNEWS/CLUE titles.

Metric	Raw LLM KG	Repaired KG
Parse success	1.000	—
Category accuracy	0.556	0.556
Keyword recall	0.178	0.178
Documents with triples	1.000	1.000
Avg. triples / doc	3.58	3.58
Invalid triple rate	0.000	0.000
Duplicate triple rate	0.000	0.000
KG quality score	100.00	100.00

H. Scope and Next Evaluation Steps

The controlled environment gives exact graph mutations and reproducible rewards, but its fixed validator inventory and regular acquisition effects make a simple heuristic effective. The learned policy’s usefulness in more varied repair settings remains to be tested with independently collected trajectories.

Generated typed rules have limited direct overlap with RuleTest-94. A validated vocabulary mapping and execution support for additional rule families are needed before those candidates can replace the registry inside the RL loop. The offline bridge provides rule-to-action traces on the designed suite; the remaining integration must evaluate learned scheduling with generated-rule arrivals and broader repair operators. Natural-defect annotation and held-out domains would then test semantic correctness and transfer. The present extraction labels support category and keyword indicators rather than gold triple accuracy.

V. Conclusion and Future Work

We presented a graph–rule co-optimization framework with a trained Double-DQN scheduler and dual-strategy text-based rule generation. The scheduling environment exposes the dependence between validator acquisition and graph repair. Its implementation includes replay, target-network updates, feasibility masks, and archived graph transitions.

With introduced violations measured after each edit, Double DQN produces no new violations on the ten evaluation scenarios, but reaches lower structural quality than the acquire-then-deficit heuristic. Its higher mean discounted return reflects the

reward’s penalty for newly introduced violations. Retrained ablations support the action mask’s final-quality contribution; independent gains from the other tested components are not established after correction.

Deletion and augmentation contribute different candidates on the same documents. At equal call budgets, augmentation produces more constraint candidates. An offline bridge now connects exact archived patterns to detection, action availability and record removal on a government-typed suite. The next step is to test learned scheduling with those generated-rule interfaces under varied rule arrivals, then evaluate factual repairs on natural defects and held-out domains. This would establish when text-derived acquisition benefits adaptive scheduling beyond the fixed registry.

Appendix A

Generation Prompts and Recorded Settings

The supplementary artifact provides literal Chinese templates exported from the prompt builders, their SHA-256 hashes, and English translations. The translations below describe the same tasks and response fields; they are not presented as the language used in the historical calls. The archive contains the parsed responses and deletion fragments, but not each fully rendered request or a complete historical decoding configuration.

A. Deletion Completion Prompt

The actual builder asks: “You are an expert in rule extraction for government-service knowledge graphs. Some phrases have been randomly removed from a service document. Read the original and the incomplete version, infer the missing rule information using the context and domain knowledge, and express it as constraints for graph validation. Return only JSON following the supplied schema.” The request includes the service item, original source, incomplete source, and deleted fragments. It asks for general types and relations rather than only instance names.

The JSON fields are `strategy`, `service_item`, `analysis`, `removed_fragments`, `missing_rule_candidates`, and `proposed_rules`. The latter contains the seven candidate categories described below. This call is made once per document; ten repeated entity masks and frequency-threshold filtering are not part of the archived runner.

B. Augmentation Expansion Prompt

The actual builder asks: “You are an expert in rule extraction for government-service knowledge graphs. Propose no more than three reasonable supplementary clauses for the document, while preserving plausibility and compliance, and infer graph-validation rules from the additions. Return only JSON following the supplied schema.” The request includes the service item and original text, and asks for reusable rather than instance-specific candidates.

Its response contains `strategy`, `service_item`, `analysis`, `added_clauses`, and `proposed_rules`. The schema categories are entity types, relationship types, forbidden type-relation

patterns, allowed type-relation patterns, hierarchy statements, geographical hierarchy patterns, and procedural statements. Both calls use the same candidate categories.

C. Defaults and Missing Metadata

The script defaults are Qwen3-32B, temperature 0.2, up to five deleted spans of at most six characters, and at most three supplementary clauses. The client does not explicitly set a completion-token cap or top-*p*. Historical per-call records contain the document identifier, service item, strategy, parsed result, and deletion fragments when applicable. They do not certify that every invocation used the current defaults. The separate 45-document extraction experiment records Qwen3.8-27B-no-thinking.

Appendix B

RuleTest-94 and Direct Execution

The suite contains 30 clean and 64 defective records: 10 hierarchy reversals, 10 absurd relations, 10 reverse-supervision cases, 30 procedural or missing-field cases, and four specialist cases. The revised scorer reads `expected_detection` directly: pass is negative, while fail and `specialist_miss` are positive. It does not derive labels by running the detector under evaluation. These are designed labels without independent annotator identities or disagreement records.

The hand-implemented family union still detects all 64 defects without flagging the 30 clean records under these stored labels. Its precision Wilson interval is [0.943, 1.000] and specificity interval [0.886, 1.000]. This family-union score is distinct from the direct execution of archived candidates reported in Section IV-C2.

Each directly compiled constraint has an identifier, allowed/forbidden kind, exact subject-type/relation/object-type pattern, and a list of original source locations. Pattern membership determines whether a record matches; allowed patterns do not define a closed-world whitelist. Contradictory permission and prohibition yield an explicit abstention. Unsupported natural-language candidates are retained without invented formal interpretations.

Appendix C

Offline Policy Study

The observation order is: four graph components; rule precision, recall and coverage; four normalized defect counts; remaining budget; and the active fractions of high- and low-precision registered modules. The action order is isolation cleanup, deduplication, relation repair, dangling-edge cleanup, deletion acquisition, augmentation acquisition, mixed acquisition, and pruning.

The new comparison uses the ten original scenarios 900000–900009. All sixty revised models (DQN, Double DQN, and four ablations) use training seeds 10000–10009, the original 250-episode budget, network dimensions, replay settings, and 260-episode exploration schedule. The graph-feature ablation zeros observation indices 0–3 and 7–10; the rule-feature ablation zeros indices 4–6 and 12–13, using zero-based indexing. Other inputs and the feasibility mask remain available. These

are explicit-feature ablations, since the mask can itself convey rule availability.

The no-mask variant removes the behavioral and Bellman masks. The no-call-penalty variant adds back the accounted acquisition penalty during training. Every evaluation uses the revised violation-identity reward, unchanged cost coefficients, and 18-step budget. Early terminal quality is carried forward to step 18 when computing trapezoidal AUC. The revision keeps the original short-trajectory AUC in its historical archive rather than mixing the two definitions.

Two-sided paired sign randomization enumerates all 2^{10} sign assignments. Confidence intervals use 10,000 whole-scenario bootstrap draws with seed 42. Holm correction covers ten comparisons: full Double DQN versus the fixed strong heuristic and each of four ablations, for final quality and common-budget AUC. The same scenarios were evaluated previously, so these follow-up comparisons do not constitute a new held-out test.

Appendix D Artifact Index

Original checkpoints, transitions, and generation records remain under their original experiment directories. The earlier offline study is in `exps/paper2_offline_revision/`. It contains frozen source hashes, forty new checkpoints and training histories, all paired evaluation transitions, equal-call candidate samples, rule provenance, explicit label checks, and publication generators. The manuscript-to-artifact audit and offline reproduction commands accompany the results. All new analyses and training runs make zero API calls.

The corrected environment, sixty retrained checkpoints, complete evaluation trajectories, shared-profile Paper1 revision, and rule bridge are in `exps/math_revision_20260925/`. Its source manifest and offline reproduction entry point accompany the results.

References

- [1] R. Das, S. Dhuliawala, M. Zaheer, L. Vilnis, I. Durugkar, A. Krishnamurthy, A. Smola, and A. McCallum, “Go for a walk and arrive at the answer: Reasoning over paths in knowledge bases using reinforcement learning,” in *International Conference on Learning Representations*, 2018.
- [2] W. Xiong, T. Hoang, and W. Y. Wang, “DeepPath: A reinforcement learning method for knowledge graph reasoning,” in *Proceedings of EMNLP*, 2017.
- [3] X. V. Lin, R. Socher, and C. Xiong, “Multi-hop knowledge graph reasoning with reward shaping,” in *Proceedings of EMNLP*, 2018.
- [4] L. Yao *et al.*, “Kg-bert: Bert for knowledge graph completion,” in *arXiv preprint arXiv:1909.03193*, 2019.
- [5] X. Wang *et al.*, “Kepler: A unified model for knowledge embedding and pre-trained language representation,” in *Transactions of the Association for Computational Linguistics*, 2021.
- [6] S. Pan, L. Luo, Y. Wang, C. Chen, J. Wang, and X. Wu, “Unifying large language models and knowledge graphs: A roadmap,” *IEEE Transactions on Knowledge and Data Engineering*, vol. 36, no. 7, pp. 3580–3599, 2024.
- [7] J. Wei *et al.*, “Chain-of-thought prompting elicits reasoning in large language models,” *Advances in Neural Information Processing Systems*, 2022.
- [8] S. Yao *et al.*, “React: Synergizing reasoning and acting in language models,” in *International Conference on Learning Representations*, 2023.
- [9] H. Knublauch and D. Kontokostas, “Shapes constraint language (shacl),” *W3C Recommendation*, 2017.

- [10] E. Prud’hommeaux *et al.*, “Shape expressions: An rdf validation and transformation language,” *Proceedings of the 10th International Conference on Semantic Systems*, 2014.
- [11] L. A. Galárraga *et al.*, “Amie: Association rule mining under incomplete evidence in ontological knowledge bases,” in *Proceedings of the 22nd International Conference on World Wide Web*, 2013.
- [12] S. Ortona *et al.*, “Rudik: Rule discovery in knowledge bases,” in *Proceedings of the VLDB Endowment*, 2018.
- [13] F. Yang *et al.*, “Differentiable learning of logical rules for knowledge base reasoning,” in *Advances in Neural Information Processing Systems*, 2017.
- [14] T.-W. Lin, G. Fierro, H. Li, T. Hong, P. Nuzzo, and A. Sangiovanni-Vincentelli, “Systematic evaluation of knowledge graph repair with large language models,” *arXiv preprint*, 2025.
- [15] V. Mnih *et al.*, “Human-level control through deep reinforcement learning,” *Nature*, vol. 518, pp. 529–533, 2015.
- [16] H. van Hasselt, A. Guez, and D. Silver, “Deep reinforcement learning with double q-learning,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, 2016, pp. 2094–2100.